BÁO CÁO THIẾT KẾ VÀ KIỂM CHỨNG BASELINE INSTRUCTION INTEGRITY TRÊN PICORV32

Mục lục

1 KIẾN TRÚC TỔNG QUAN **4**

1.1 Phân định phạm vi kiến trúc và cấu hình thiết kế 4

1.2 Kiến trúc mức logic của hệ thống 5

1.3 Tổ chức hiện thực và môi trường triển khai 7

2 KIẾN TRÚC TOP-LEVEL **8**

2.1 Mô tả tổng quan kiến trúc Top-Level 8

2.2 Chi tiết các khối chức năng trong kiến trúc Top-Level 9

2.3 Các luồng tín hiệu và giao tiếp bus liên kết 9

3 THIẾT KẾ KHỐI CHỨC NĂNG **10**

3.1 Khối Authenticated Memory Wrapper 11

3.1.1 Chức năng 11

3.1.2 Thuật toán và phép toán hiện thực 12

3.1.3 Thiết kế phần cứng 14

3.2 Khối Ascon-128a 18

3.2.1 Chức năng 18

3.2.2 Thiết kế phần cứng 19

3.3 Khối System Memory 23

3.3.1 Chức năng 23

3.3.2 Thuật toán và nguyên lý vận hành 23

3.3.3 Thiết kế phần cứng 25

3.4 Khối Trusted Key and Context 25

3.4.1 Chức năng 25

3.4.2 Tổ chức khóa và ngữ cảnh 25

3.4.3 Thiết kế phần cứng 26

3.4.4 Thiết kế phần cứng 27

3.5 Lõi PicoRV32 và giao diện tích hợp 27

4 Lõi PicoRV32 và giao diện tích hợp 27

4.1 Chức năng 27

4.2 Cấu hình sử dụng 27

4.3 Giao diện và hành vi khi xác thực thất bại 28

5 MÔ HÌNH KIỂM CHỨNG 30

5.1 Mô hình kiểm chứng tổng quan 30

5.2 Mô hình tham chiếu và luồng tạo code/tag 30

5.3 Testbench, monitor và cơ chế đối sánh 30

5.4 Kế hoạch kiểm chứng và tiêu chí đánh giá 30

6 KIỂM CHỨNG 31

6.1 Kiểm chứng các khối con của Wrapper 31

6.2 Kiểm chứng khối Ascon-128a 31

6.3 Kiểm chứng khối System Memory 31

6.4 Kiểm chứng Authenticated Memory Wrapper 31

6.5 Kiểm chứng tích hợp PicoRV32 và các kịch bản tấn công 31

6.6 Tổng hợp kết quả kiểm chứng 31

7 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ 32

7.1 Môi trường và cấu hình thực nghiệm 32

7.2 Kết quả chức năng và phát hiện sửa đổi mã lệnh 32

7.3 Hiệu năng và chi phí xác thực 32

7.4 Ảnh hưởng của kích thước block và độ dài tag 32

7.5 Tài nguyên, timing và công suất 32

7.6 Đánh giá tổng hợp và giới hạn 32

Danh sách hình vẽ

1 Kiến trúc tổng quan hệ thống Instruction Integrity 7

2 Kiến trúc Top-level 8

3 Kiến trúc giao tiếp phần cứng top-level 10

4 Kiến trúc Authenticated Memory 11
5 Máy trạng thái FSM 18
6 Sơ đồ khối Ascon-128a 21
7 Máy trạng thái FSM của khối `ascon_controller` 22
8 Sơ đồ System Memory 25

Danh sách bảng

1 Ánh xạ khối kiến trúc sang mã nguồn RTL 8
2 Cổng vào/ra của module top-level 9
3 Kích thước AD và số beat theo G 13
4 Giao diện tín hiệu khối `authenticated_memory_wrapper` 14
5 Giao diện tín hiệu của khối `ascon_top` 19
6 Bản đồ bộ nhớ trong RTL hiện tại 23
7 Hành vi của `system_memory` theo loại truy cập 24
8 Tổ chức vector `CFG` 224 bit 26
9 Giao diện tín hiệu của khối `trusted_key_and_context` 26
10 Tham số của khối `trusted_key_and_context` 26
11 Tham số PicoRV32 trong `auth_soc_top` 28
12 Giao diện bộ nhớ của PicoRV32 nối vào wrapper 29
13 Tổng hợp kết quả kiểm chứng 31
14 So sánh cấu hình trên cùng firmware 32

1 KIẾN TRÚC TỔNG QUAN

1.1 Phân định phạm vi kiến trúc và cấu hình thiết kế

Đề tài nghiên cứu kỹ thuật xác thực và bảo vệ tính toàn vẹn của lệnh (Instruction Integrity) nhằm bảo vệ lõi xử lý RISC-V chống lại các cuộc tấn công sửa đổi trái phép mã lệnh. Để đánh giá chính xác sự đánh đổi giữa hiệu năng, chi phí diện tích và mức độ an toàn, thiết kế được xây dựng theo phương pháp tiếp cận từ dưới lên thông qua các cấu hình:

1. **Kiến trúc Baseline (Gốc):** Bao gồm lõi xử lý RISC-V PicoRV32, khối ASCON-128A dùng để tạo mã xác thực (authentication tag), khối Authenticated Memory thực hiện xác thực tag và khối System Memory lưu trữ mã lệnh (code), dữ liệu (data) cùng các tag tương ứng. Ở cấu hình này, ASCON-128A tạo tag 128-bit cho mỗi block dữ liệu, trong khi Authenticated Memory hoạt động ở chế độ xác thực theo từng block gồm 4 word (128-bit), bằng cách kiểm tra tag tương ứng được lưu trong System Memory trước khi chuyển lệnh đến CPU. Mục đích của Baseline là thiết lập mốc tham chiếu (ground-truth) về độ trễ, số chu kỳ thực thi lệnh (CPI), chi phí diện tích phần cứng và khả năng phát hiện các hành vi sửa đổi trái phép mã lệnh.

Các tham số chính của cấu hình tích hợp gồm:

- G: số word 32 bit trong một block; mặc định $G = 4$.
- T: độ dài tag được so sánh; mặc định $T = 128$ bit.
- `FAULT_HALT`: lựa chọn hành vi khi xác thực thất bại.

2. **Kiến trúc Dynamic Authorization (Moving Target Defense):** Kiến trúc này giữ nguyên lõi RISC-V PicoRV32, khối ASCON-128A và Authenticated Memory của Baseline, đồng thời bổ sung cơ chế cho phép trạng thái xác thực (authorization state) thay đổi theo thời gian trong khi CPU vẫn chạy. Mã lệnh được chia thành R vùng (region); mỗi vùng có protection version v riêng và khóa riêng $K(r, v)$ được dẫn xuất bên trong khối tin cậy (trusted key and context). Tag của mỗi block được tính trên thông điệp gồm nội dung, địa chỉ block, `region_id` và `version`, nhờ đó một cặp (mã lệnh, tag) thuộc version cũ không còn được chấp nhận cho các lần fetch mới.

   Việc chuyển version được thực hiện bởi một *churn unit* đặt ngoài lõi, theo cơ chế threshold register của Morpheus (ASPLOS 2019, §4.4), qua ba pha `prepare` →

`publish` → `retire`. Churn unit quét lần lượt từng block của một vùng: xác thực block bằng khóa cũ, tính lại tag trên chính buffer vừa xác thực bằng khóa mới, ghi tag mới rồi mới tăng thanh ghi ngưỡng *thr*. Các block có địa chỉ nhỏ hơn *thr* dùng context mới, các block còn lại dùng context cũ. Fetch vào đúng block đang ở ngưỡng phải chờ, và churn unit được ưu tiên sau tối đa một job xác thực đang chạy. Mỗi fetch được ghim (pin) vào một context duy nhất từ lúc được nhận cho đến khi kết thúc, nên không có fetch nào dùng lẫn khóa cũ và khóa mới. Vì mã lệnh là plaintext và tag chỉ nằm trong bộ nhớ metadata, không giá trị nào bên trong lõi phụ thuộc vào khóa; do đó không cần flush pipeline hay gắn domain tag. Khi sweep hoàn tất, khóa cũ được hủy (retire) và hệ thống không bao giờ quay lại version cũ.

Churn được kích hoạt bởi timer tin cậy. Cùng churn unit này còn có chế độ *stop-the-world* (tạm dừng core, re-authenticate toàn vùng rồi mới tiếp tục), dùng làm cấu hình đối chứng để so sánh thời gian dừng và chi phí với live churn. Mục đích của kiến trúc là đánh giá: (i) khả năng giảm replay, transplantation và giới hạn thời gian, phạm vi của khóa bị lộ; (ii) tính nhất quán của context khi chuyển version lúc đang thực thi; (iii) chi phí về stall, thời gian churn, metadata và tài nguyên phần cứng so với Baseline.

Các tham số chính bổ sung so với Baseline gồm:

- R: số vùng mã (region), mỗi vùng có `region_id` riêng;
- v: protection version của một vùng, được đưa vào thông điệp MAC;
- $K(r, v)$: khóa của vùng r ở version v, dẫn xuất trong khối tin cậy;
- *thr*: thanh ghi ngưỡng của mỗi vùng, xác định block nào đã mang tag mới;
- T_{churn}: chu kỳ kích hoạt churn của timer tin cậy;
- `CHURN_MODE`: `THRESHOLD_SWEEP` (live churn) hoặc `STOP_WORLD` (đối chứng);
- `FAULT_HALT`: hành vi khi xác thực thất bại, kể cả khi lỗi xảy ra giữa lúc sweep; không fallback về khóa cũ.

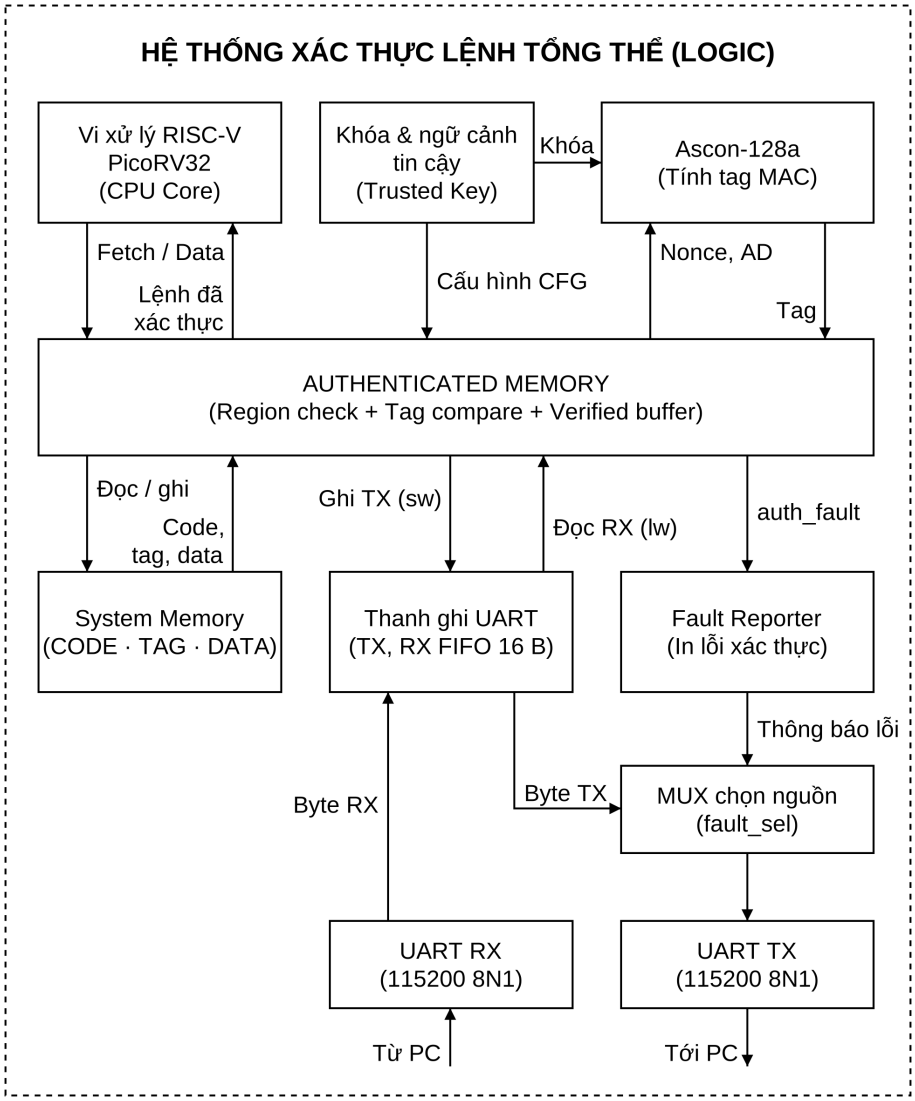
1.2 Kiến trúc mức logic của hệ thống

Hệ thống gồm năm thành phần: PicoRV32, Authenticated Memory Wrapper, System Memory, Ascon-128a và Trusted Key and Context.

- **PicoRV32:** lõi RISC-V 32-bit mã nguồn mở do Claire Wolf phát triển và được duy trì trong tổ chức YosysHQ [1]. Thiết kế được dùng nguyên bản từ `YosysHQ/picorv32`, không sửa đổi bên trong lõi. Lõi giao tiếp với bộ nhớ qua *native memory interface*, trong đó mỗi thời điểm chỉ có một giao dịch bộ nhớ (`mem_valid` giữ mức cao cho đến khi có `mem_ready`).

- **Ascon-128a:** thuật toán mã hóa xác thực nhẹ (lightweight AEAD) thuộc họ Ascon, do Dobraunig, Eichlseder, Mendel và Schläffer thiết kế [2]. Họ Ascon được chọn làm chuẩn mã hóa nhẹ của NIST [3]. Trong đề tài, khối này được dùng để tạo và kiểm tra tag xác thực 128-bit cho từng block mã lệnh. Dữ liệu cần bảo vệ được đưa vào làm associated data (địa chỉ block và các word mã lệnh), do mã lệnh vẫn ở dạng plaintext và chỉ cần bảo vệ tính toàn vẹn, không cần giữ bí mật.
- **Authenticated Memory Wrapper:** khối điều phối chính, nằm giữa PicoRV32 và System Memory. Wrapper chặn yêu cầu bộ nhớ của lõi, chốt địa chỉ và loại truy cập, đọc các word của block cùng tag tương ứng từ System Memory, đóng gói chúng thành thông điệp và gửi tới Ascon-128a để tính tag. Wrapper so sánh tag tính được với tag lưu trữ: nếu khớp, lệnh được chuyển cho CPU; nếu không khớp, wrapper kích hoạt hành vi `FAULT_HALT` và không trả lệnh cho lõi.
- **System Memory:** bộ nhớ chính lưu mã lệnh (code), dữ liệu (data) và các tag tương ứng của từng block. Đây là vùng nằm ngoài ranh giới tin cậy, nên nội dung của nó được giả định có thể bị attacker đọc hoặc sửa; mọi dữ liệu lấy từ đây đều phải qua bước xác thực của wrapper trước khi tới CPU.
- **Trusted Key and Context:** khối tin cậy lưu khóa bí mật và các tham số cấu hình toàn cục (kích thước block G, độ dài tag T, định danh của firmware image dùng để tạo nonce). Khối này cấp khóa và ngữ cảnh trực tiếp cho Ascon-128a, không đi qua bus hay System Memory, nên khóa không bao giờ xuất hiện ở vùng không tin cậy.

Wrapper điều phối quá trình xác thực. Khóa được cấp trực tiếp từ khối tin cậy tới Ascon.



Hình 1: Kiến trúc tổng quan hệ thống Instruction Integrity

1.3 Tổ chức hiện thực và môi trường triển khai

Hình 1 được hiện thực hoàn toàn trên một FPGA Gowin GW2AR-18 (Sipeed Tang Nano 20K). Bảng 1 ánh xạ từng khối của kiến trúc mức logic sang mã nguồn RTL; ngoài năm khối đã mô tả, hệ thống có thêm khối UART là ngoại vi duy nhất, dùng để quan sát đầu ra và đọc/ghi bộ nhớ khi gỡ lỗi.

Môi trường công cụ và thiết bị:

- **Mô phỏng RTL:** ModelSIM; kết quả báo cáo bằng số chu kỳ xung nhịp.
- **Biên dịch firmware:** xPack GNU RISC-V Embedded GCC 15.2.0-1 (`riscv-none-elf-gcc`, chạy trên Windows) [`-march`, `-mabi`]; image mã lệnh và tag được tạo trước rồi nạp vào System Memory.

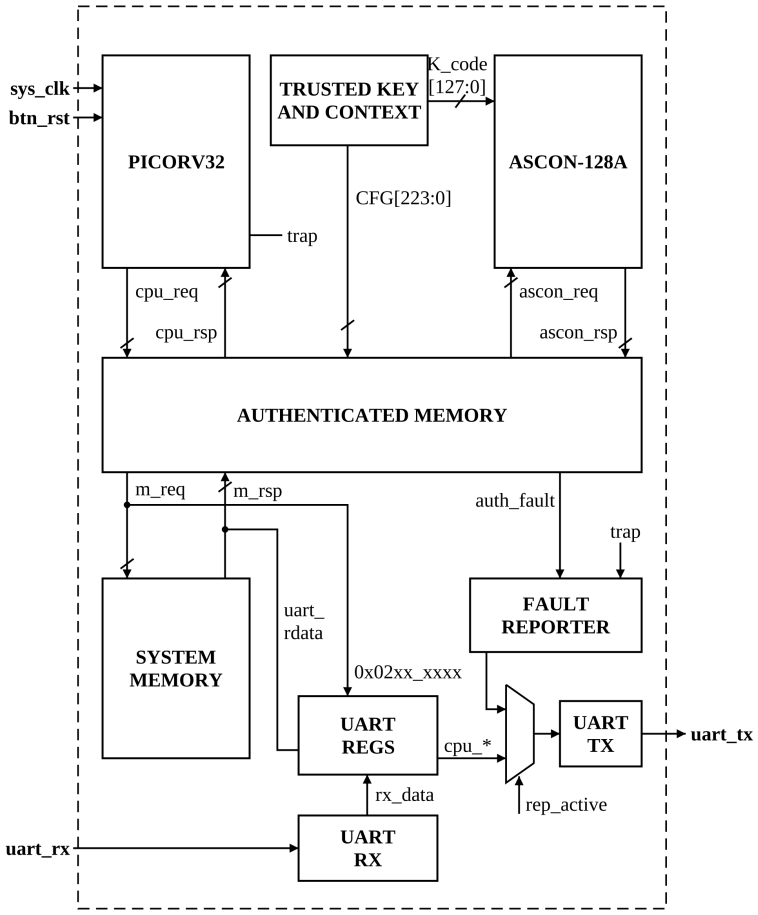
Bảng 1: Ánh xạ khối kiến trúc sang mã nguồn RTL

Khối	**RTL**
Top-level SoC (gồm UART)	`soc/auth_soc_top.v`
PicoRV32	`auth_sim/picorv32.v` (nguyên bản, không sửa)
Authenticated Memory Wrapper	`authenticated_memory/`
Ascon-128a	`ascon/`
System Memory	`memory/`
Trusted Key and Context	`trusted_key_and_context`

- **Tổng hợp và triển khai:** Gowin EDA [19.11 Education] trên Tang Nano 20K, giao tiếp PC qua mạch UART–USB CP2102.

2 KIẾN TRÚC TOP-LEVEL

2.1 Mô tả tổng quan kiến trúc Top-Level



Hình 2: Kiến trúc Top-level

Bảng 2: Cổng vào/ra của module top-level

Tín hiệu	Type	Width (bit)	Pin	Chức năng
sys_clk	input	1	4	Xung nhịp hệ thống
btn_rst	input	1	88	Nút reset active low
uart_rx	input	1	42	Nhận dữ liệu UART từ máy tính (TXD của CP2102)
uart_tx	Ra	1	41	Truyền dữ liệu UART tới máy tính (RXD của CP2102)

2.2 Chi tiết các khối chức năng trong kiến trúc Top-Level

- picorv32: thực thi chương trình và phát giao dịch bộ nhớ.
- authenticated_memory_wrapper: phân loại truy cập, kiểm tra vùng địa chỉ, xác thực block và điều khiển phản hồi.
- system_memory: lưu code, tag và dữ liệu.
- ascon_top: tính tag từ khóa, nonce và dữ liệu AD.
- trusted_key_and_context: cấp khóa và cấu hình tin cậy.
- uart_tx: đóng khung và phát dữ liệu nối tiếp ra chân TX.
- uart_rx: lấy mẫu chân RX và ghép thành byte.

2.3 Các luồng tín hiệu và giao tiếp bus liên kết

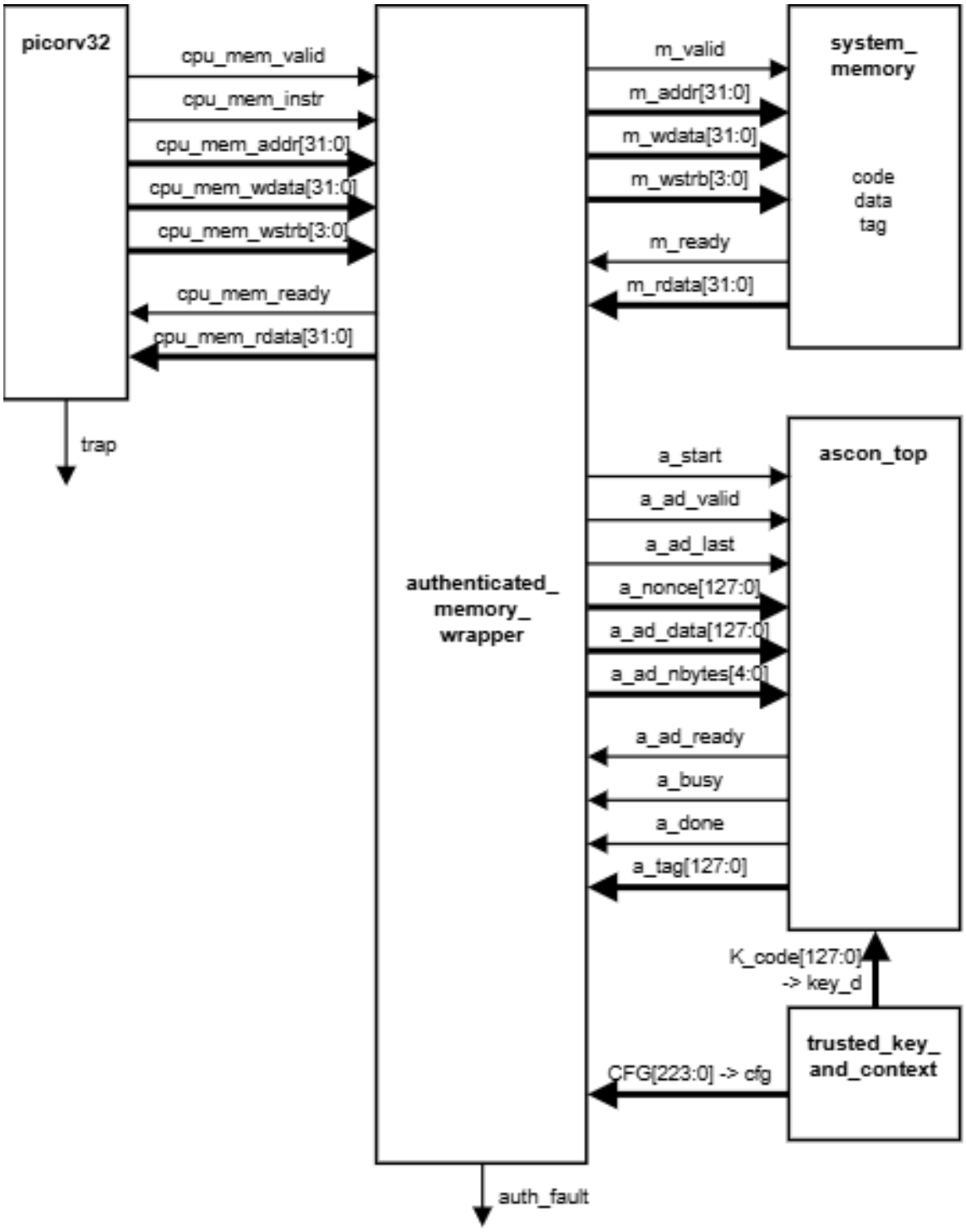
Luồng CPU–Wrapper cpu_mem_valid, cpu_mem_instr, cpu_mem_addr, cpu_mem_wdata, cpu_mem_wstrb, cpu_mem_ready và cpu_mem_rdata.

Luồng Wrapper–System Memory m_valid, m_addr, m_wdata, m_wstrb, m_ready và m_rdata.

Luồng Wrapper–Ascon a_start, a_nonce, a_ad_data, a_ad_nbytes, a_ad_last, a_ad_valid, a_ad_ready, a_busy, a_done và a_tag.

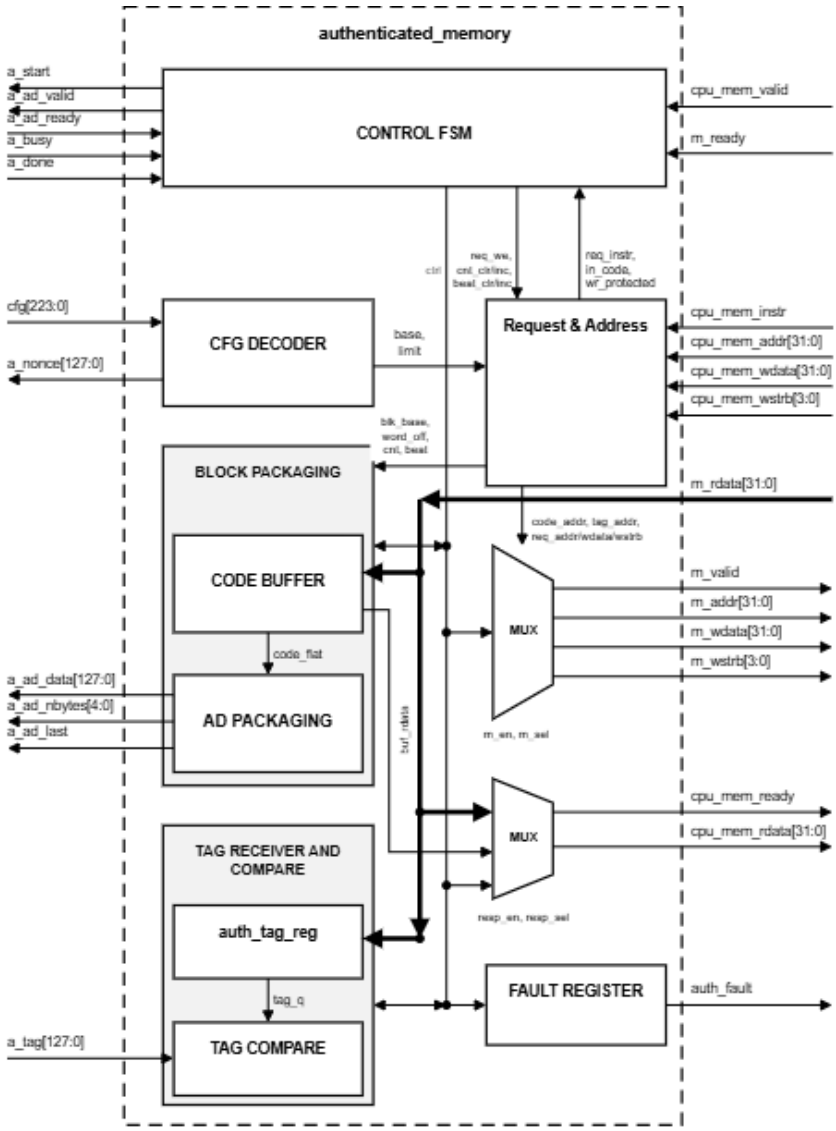
Luồng khóa, cấu hình và báo lỗi K_code đi từ khối tin cậy tới Ascon; CFG đi tới wrapper; auth_fault là cờ lỗi sticky và trap là ngõ ra của PicoRV32.

3 THIẾT KẾ KHỐI CHỨC NĂNG



Hình 3: Kiến trúc giao tiếp phần cứng top-level

3.1 Khối Authenticated Memory Wrapper



Hình 4: Kiến trúc Authenticated Memory

3.1.1 Chức năng

Khối `authenticated_memory_wrapper` nằm giữa lõi PicoRV32 và bộ nhớ hệ thống. Nó chặn mọi giao dịch bộ nhớ của CPU, xác thực khối lệnh (block) bằng thẻ xác thực (tag) do Ascon-128a tính, và chỉ trả lệnh về cho CPU khi tag hợp lệ (Hình 4). Các chức năng chính gồm:

- **Chặn và lưu yêu cầu của CPU.** Khi `cpu_mem_valid` lên mức cao, register bên trong chốt `cpu_mem_instr`, `cpu_mem_addr`, `cpu_mem_wdata` và `cpu_mem_wstrb`. Nhờ đó các tín hiệu của CPU không bị thay đổi trong lúc wrapper thực hiện chuỗi truy cập nhiều chu kỳ.
- **Phân loại địa chỉ.** `auth_addr_check` so địa chỉ với `code_base`, `code_limit`, `tag_base`, `tag_limit` (do `auth_cfg_decode` giải mã từ `cfg`). Kết quả là hai cờ: `in_code` (truy cập vùng

mã được bảo vệ) và `wr_protected` (ghi vào vùng mã hoặc vùng tag). Truy cập ngoài vùng bảo vệ, như dữ liệu và UART, được chuyển thẳng xuống bộ nhớ qua trạng thái `MEMORY_ACCESS`.

- **Sinh địa chỉ theo block.** `auth_addr_gen` tính `blk_base` (địa chỉ đầu block chứa lệnh cần lấy), `word_off` (vị trí từ lệnh trong block), `code_addr` và `tag_addr` (địa chỉ từng từ của block và của tag tương ứng). Hai bộ đếm `auth_counter` đếm từ trong block (`cnt`) và đếm beat gửi sang Ascon (`beat`).
- **Đóng gói và gửi block đi tính tag .** Wrapper đọc đủ G từ của block vào `auth_code_buffer`. `auth_ad_packer` ghép `blk_base` và nội dung block (`code_flat`) thành các beat 128 bit `a_ad_data`, kèm `a_ad_nbytes` và `a_ad_last`. Nonce `a_nonce` lấy từ `program_id` và `image_id`. Wrapper phát `a_start` và `a_ad_valid` theo giao thức bắt tay `a_ad_ready` để đưa dữ liệu sang Ascon.
- **Nhận tag và so sánh .** Tag tham chiếu của block được đọc từ vùng tag trong bộ nhớ và lưu vào `auth_tag_reg` (`tag_q`). Khi Ascon báo `a_done`, `auth_tag_compare` so `a_tag` với `tag_q` và cho ra `tag_ok`.
- **Quyết định cho phép hay chặn.** Nếu `tag_ok` bằng 1, block được đánh dấu đã xác thực trong `auth_code_buffer` (`buf_set`) và lệnh được trả về CPU. Nếu sai tag, fetch ngoài vùng mã hoặc địa chỉ không hợp lệ, hoặc ghi vào vùng code/tag, wrapper vào trạng thái `FAULT`: bật cờ `auth_fault` (giữ đến khi reset), xóa block đang lưu trong buffer và trả lệnh bằng 0 để PicoRV32 vào trap (khi `FAULT_HALT`=1 thì giữ nguyên trạng thái, CPU bị treo).
- **Tận dụng block đã xác thực.** Nếu lệnh nằm trong block đã xác thực (`buf_hit`), wrapper trả lệnh ngay từ buffer mà không truy cập lại bộ nhớ hay chạy lại Ascon. Nhờ đó các lệnh liên tiếp trong cùng block chỉ phải xác thực một lần.
- **Hợp nhất đường truy cập bộ nhớ và đường trả lời.** `auth_mem_mux` chọn địa chỉ gửi xuống bộ nhớ là địa chỉ mã (`code_addr`), địa chỉ tag (`tag_addr`) hoặc yêu cầu gốc của CPU. `auth_resp_mux` chọn dữ liệu trả cho CPU từ buffer, từ bộ nhớ hoặc bằng 0, đồng thời tạo `cpu_mem_ready`.
- **Điều phối.** `auth_controller` là máy trạng thái điều khiển toàn bộ chuỗi trên: `IDLE`, `CHECK_REQUEST`, `READ_CODE`, `READ_TAG`, `START_ASCON`, `SEND_AD`, `WAIT_ASCON`, `COMPARE`, `RESPONSE`, `FAULT` và `MEMORY_ACCESS`.

3.1.2 Thuật toán và phép toán hiện thực

Wrapper được tham số hóa bởi G (số từ 32 bit trong một block, mặc định $G = 4$) và T (số bit tag, mặc định $T = 128$). Kích thước block là $B = 4G$ byte, và tag chiếm $T/32$ từ.

Kiểm tra địa chỉ. Với địa chỉ `addr` và `wstrb` của CPU:

$$in_code_rng = (\texttt{code_base} \le \texttt{addr} \le \texttt{code_limit}), \tag{1}$$

$$in_code = (\texttt{addr}[1{:}0] = 00) \ \wedge \ in_code_rng, \tag{2}$$

$$wr_protected = (\texttt{wstrb} \neq 0) \ \wedge \ (in_code_rng \ \vee \ in_tag_rng). \tag{3}$$

Vùng tag được suy ra từ vùng mã, mỗi block mã ứng với một tag:

$$\texttt{tag_limit} = \texttt{tag_base} + \left(\left\lfloor \frac{\texttt{code_limit} - \texttt{code_base}}{B} \right\rfloor + 1 \right) \cdot \frac{T}{8} - 1, \tag{4}$$

với phép chia B và nhân $T/8$ thực hiện bằng dịch bit ($\log_2 B$ và $\log_2(T/8)$).

Sinh địa chỉ theo block. Với $i = \texttt{cnt}$ là chỉ số từ đang đọc:

$$\texttt{blk_base} = \texttt{addr} \ \& \ \sim (B-1), \tag{5}$$

$$\texttt{word_off} = (\texttt{addr} \ \& \ (B-1)) \gg 2, \tag{6}$$

$$\texttt{code_addr}_i = \texttt{blk_base} + 4i, \tag{7}$$

$$blk_index = (\texttt{blk_base} - \texttt{code_base}) \gg \log_2 B, \tag{8}$$

$$\texttt{tag_addr}_i = \texttt{tag_base} + (blk_index \ll \log_2(T/8)) + 4i. \tag{9}$$

Đóng gói dữ liệu liên kết (AD) gửi Ascon. Dữ liệu cần xác thực là địa chỉ đầu block nối với nội dung block, độn bằng 0 cho tròn beat 128 bit:

$$\mathrm{AD} = \texttt{blk_base} \,\|\, W_0 \,\|\, W_1 \,\|\, \cdots \,\|\, W_{G-1} \,\|\, 0 \cdots 0, \qquad |\mathrm{AD}| = 4 + 4G \text{ byte}. \tag{10}$$

Số beat là $N = \lceil (4+4G)/16 \rceil$; các beat đầu gửi đủ 16 byte, beat cuối gửi $(4+4G) - 16(N-1)$ byte (`a_ad_nbytes`), và `a_ad_last`$= (\texttt{beat} = N-1)$. Việc gắn `blk_base` vào AD ràng buộc tag với vị trí của block, nên một block hợp lệ bị chép sang địa chỉ khác sẽ không qua được bước so sánh. Nonce gửi Ascon là $\texttt{a_nonce} = \texttt{program_id} \,\|\, \texttt{image_id}$ (128 bit), lấy từ `cfg`.

Bảng 3: Kích thước AD và số beat theo G

G	$\lvert\mathrm{AD}\rvert$ (byte)	Số beat N	Byte beat cuối
1	8	1	8
2	12	1	12
4	20	2	4
8	36	3	4

So sánh tag. Tag tham chiếu `tag_q` được đọc từ bộ nhớ và nạp từng từ 32 bit vào thanh ghi T bit. Sau khi Ascon báo `a_done`:

$$\texttt{tag_ok} = (\texttt{a_tag}[127 : 128 - T] = \texttt{tag_q}). \tag{11}$$

Thuật toán xử lý một yêu cầu của CPU.

1. `IDLE`: chờ `cpu_mem_valid`, chốt yêu cầu vào `auth_req_reg`.
2. `CHECK_REQUEST`: xóa `cnt` và `beat`.
   - Nếu là fetch: ngoài vùng mã hoặc sai căn hàng thì sang `FAULT`; nếu `buf_hit` thì sang `RESPONSE`; ngược lại xóa hiệu lực buffer và sang `READ_CODE`.
   - Nếu là truy cập dữ liệu: ghi vào vùng mã hoặc tag (*wr_protected*) thì sang `FAULT`; ngược lại sang `MEMORY_ACCESS`.
3. `READ_CODE`: đọc G từ của block (`m_sel`=CODE) vào buffer.
4. `READ_TAG`: đọc $T/32$ từ tag (`m_sel`=TAG) vào `auth_tag_reg`.
5. `START_ASCON`: khi Ascon rảnh (`a_busy`=0), phát `a_start`.
6. `SEND_AD`: gửi lần lượt N beat AD theo bắt tay `a_ad_valid`/`a_ad_ready`.
7. `WAIT_ASCON`: chờ `a_done`.
8. `COMPARE`: nếu `tag_ok` thì đặt hiệu lực cho buffer (lưu `blk_base`) và sang `RESPONSE`; ngược lại sang `FAULT`.
9. `RESPONSE`: trả từ `word_off` của buffer cho CPU rồi về `IDLE`.
10. `MEMORY_ACCESS`: chuyển nguyên giao dịch dữ liệu xuống bộ nhớ, trả kết quả cho CPU.
11. `FAULT`: bật cờ lỗi, xóa buffer, trả 0 cho CPU (hoặc treo nếu `FAULT_HALT`=1).

Khi miss, wrapper đọc $G + T/32$ từ từ bộ nhớ trước khi xác thực. Khi hit, lệnh được trả ngay từ buffer nên chi phí xác thực chỉ phát sinh một lần cho mỗi block.

3.1.3 Thiết kế phần cứng

Port map

Bảng 4: Giao diện tín hiệu khối `authenticated_memory_wrapper`

Port	**Dir**	**Width**	**Data Type**	**Action level**	**Mô tả hành vi**
Xung nhịp và reset					

Còn tiếp ở trang sau

Port	Dir	Width	Data Type	Action level	Mô tả hành vi
clk	IN	1	Logic	Rising edge	Xung nhịp của toàn khối.
rst_n	IN	1	Logic	Low	Reset bất đồng bộ, đưa FSM về IDLE và xóa buffer, tag, cờ lỗi.
Giao tiếp CPU (PicoRV32)					
cpu_mem_valid	IN	1	Logic	High	Yêu cầu giao dịch bộ nhớ từ CPU; FSM chốt yêu cầu khi ở IDLE.
cpu_mem_instr	IN	1	Logic	High	Giao dịch là fetch lệnh (phải xác thực); mức thấp là truy cập dữ liệu.
cpu_mem_addr	IN	32	Unsigned	-	Địa chỉ giao dịch; được kiểm tra vùng, căn hàng 4 byte và tách thành địa chỉ block.
cpu_mem_wdata	IN	32	Logic	-	Dữ liệu ghi từ CPU, chỉ chuyển xuống bộ nhớ khi truy cập dữ liệu.
cpu_mem_wstrb	IN	4	Logic	High	Mặt nạ byte ghi; bằng 0 là đọc. Ghi vào vùng mã hoặc tag bị coi là lỗi.
cpu_mem_ready	OUT	1	Logic	High	Báo giao dịch hoàn tất; chỉ phát sau khi lệnh được xác thực hoặc truy cập dữ liệu xong.
cpu_mem_rdata	OUT	32	Logic	-	Dữ liệu trả CPU: từ buffer, từ bộ nhớ, hoặc 0 khi lỗi.
Giao tiếp bộ nhớ hệ thống					
m_ready	IN	1	Logic	High	Bộ nhớ hoàn tất một lần đọc hoặc ghi; FSM chuyển bước khi nhận được.
m_rdata	IN	32	Logic	-	Dữ liệu đọc: từ mã, từ tag hoặc dữ liệu của CPU.
m_valid	OUT	1	Logic	High	Yêu cầu truy cập bộ nhớ trong các trạng thái đọc mã, đọc tag và truy cập dữ liệu.

Còn tiếp ở trang sau

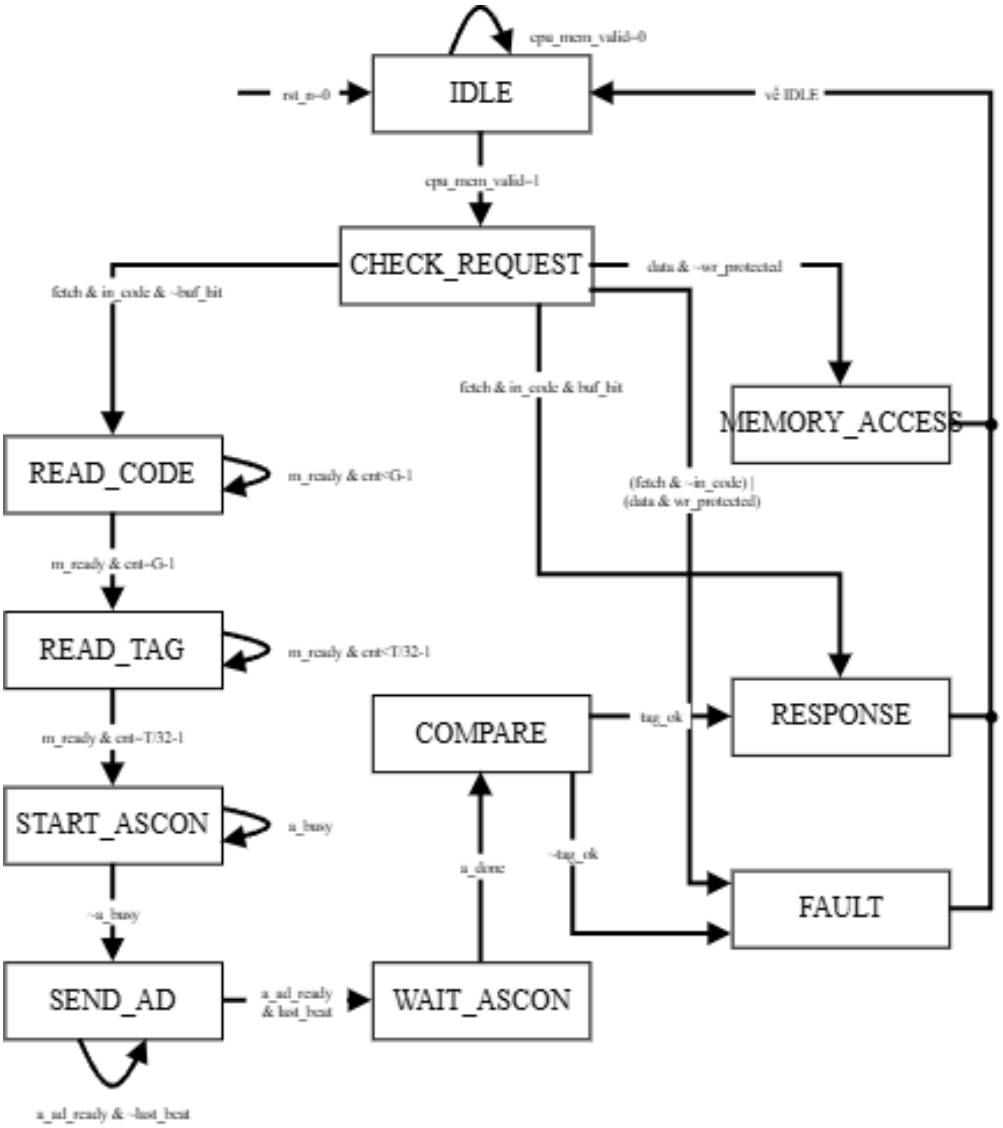
Port	Dir	Width	Data Type	Action level	Mô tả hành vi
`m_addr`	OUT	32	Unsigned	-	Địa chỉ mã, địa chỉ tag hoặc địa chỉ của CPU tùy trạng thái.
`m_wdata`	OUT	32	Logic	-	Dữ liệu ghi; bằng 0 khi đọc mã hoặc tag.
`m_wstrb`	OUT	4	Logic	High	Mặt nạ byte ghi; chỉ khác 0 khi chuyển giao dịch ghi dữ liệu của CPU.
Giao tiếp Ascon-128a					
`a_start`	OUT	1	Logic	High	Khởi động Ascon; giữ ở trạng thái `START_ASCON`.
`a_nonce`	OUT	128	Logic	-	Nonce ghép từ `program_id` và `image_id` lấy từ `cfg`.
`a_ad_valid`	OUT	1	Logic	High	Beat AD hợp lệ; giữ ở trạng thái `SEND_AD`.
`a_ad_data`	OUT	128	Logic	-	Beat AD gồm địa chỉ block và nội dung block, độn 0 cho tròn beat.
`a_ad_nbytes`	OUT	5	Unsigned	-	Số byte hợp lệ của beat: 16, hoặc ít hơn ở beat cuối.
`a_ad_last`	OUT	1	Logic	High	Đánh dấu beat AD cuối cùng.
`a_ad_ready`	IN	1	Logic	High	Ascon nhận được beat AD; FSM chuyển sang beat kế hoặc sang `WAIT_ASCON`.
`a_busy`	IN	1	Logic	High	Ascon đang bận; FSM chờ mức thấp trước khi khởi động.
`a_done`	IN	1	Logic	High	Ascon tính xong tag; FSM chuyển sang so sánh.
`a_tag`	IN	128	Logic	-	Tag do Ascon tính; T bit cao được so với tag đọc từ bộ nhớ.
Cấu hình tin cậy và báo lỗi					

Còn tiếp ở trang sau

Port	Dir	Width	Data Type	Action level	Mô tả hành vi
`cfg`	IN	224	Logic	-	Gồm `program_id` (64), `image_id` (64), `code_base` (32), `code_limit` (32), `tag_base` (32).
`auth_fault`	OUT	1	Logic	High	Cờ lỗi xác thực (tag sai, fetch ngoài vùng mã, ghi vào vùng được bảo vệ); giữ đến khi reset.

Datapath và các khối con *[Cần bổ sung: Mô tả `auth_req_reg`, `auth_cfg_decode`, `auth_addr_check`, `auth_addr_gen`, `auth_counter`, `auth_code_buffer`, `auth_tag_reg`, `auth_ad_packer`, `auth_tag_compare`, `auth_mem_mux`, `auth_resp_mux`, `auth_fault_reg` và `auth_perf_counters`.]*

Máy trạng thái điều khiển *[Cần bổ sung: Mô tả 11 trạng thái của `auth_controller`: IDLE, CHECK_REQUEST, READ_CODE, READ_TAG, START_ASCON, SEND_AD, WAIT_ASCON COMPARE, RESPONSE, FAULT, MEMORY_ACCESS.]*



Hình 5: Máy trạng thái FSM

Chu kỳ, buffer và xử lý lỗi

3.2 Khối Ascon-128a

3.2.1 Chức năng

Khối `ascon_top` tính tag Ascon-128a v1.2. rõ luôn rỗng, nên đầu ra duy nhất là tag 128 bit cho phần AD. Khối nhận khóa mã `K_code` (`key_d`) từ khối khóa tin cậy, nonce `a_nonce` (`nonce_d`) và dữ liệu AD từ wrapper. Thuật toán gồm bốn bước trên trạng thái 320 bit (năm thanh ghi 64 bit):

1. **Khởi tạo:** nạp trạng thái ban đầu từ IV, khóa và nonce, chạy hoán vị p^{12}, rồi XOR khóa vào hai thanh ghi cuối.

2. **Hấp thụ AD:** mỗi beat 128 bit được độn (ở beat cuối) và XOR vào phần tốc độ của trạng thái, sau đó chạy hoán vị p^8.

3. **Phân tách miền:** độn bản rõ rỗng, XOR khóa và đặt bit phân tách miền.

4. **Kết thúc:** chạy p^{12}, rồi tag $= \{S_3 \oplus K_0,\ S_4 \oplus K_1\}$.

Hai bước khởi tạo và kết thúc dùng chung một khối hoán vị p^{12} để tiết kiệm tài nguyên.

Giao tiếp với wrapper theo hai tín hiệu điều khiển và một bắt tay:

- `start` bắt đầu một phép tính; `busy` báo khối đang bận.
- `ad_valid`/`ad_ready` bắt tay từng beat AD, kèm `ad_nbytes` (số byte hợp lệ) và `ad_last` (beat cuối).
- `done` báo tag đã sẵn sàng trên `tag_out`, và tag được giữ ổn định cho đến phép tính kế tiếp để wrapper so sánh.

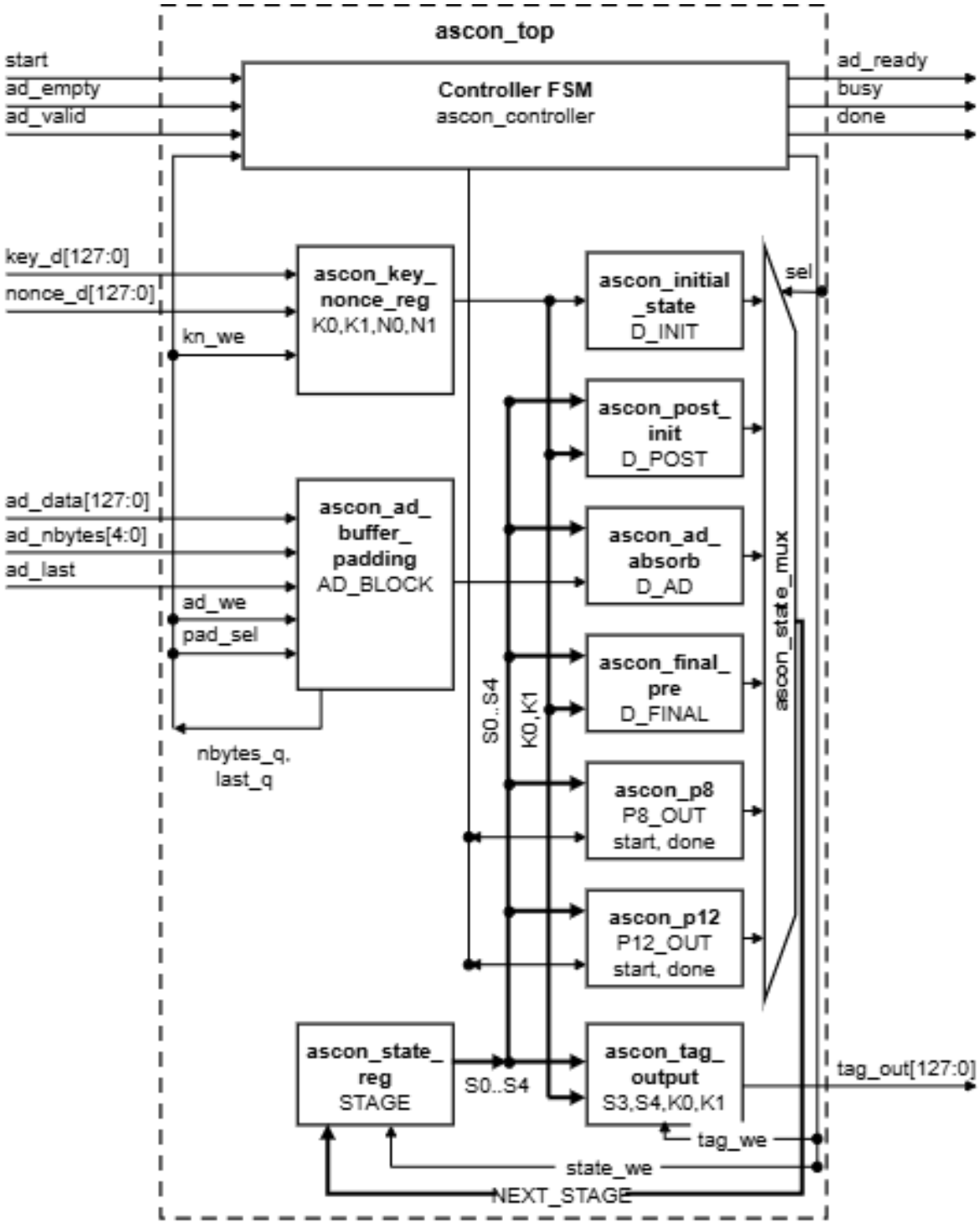
3.2.2 Thiết kế phần cứng

Port map

Bảng 5: Giao diện tín hiệu của khối `ascon_top`

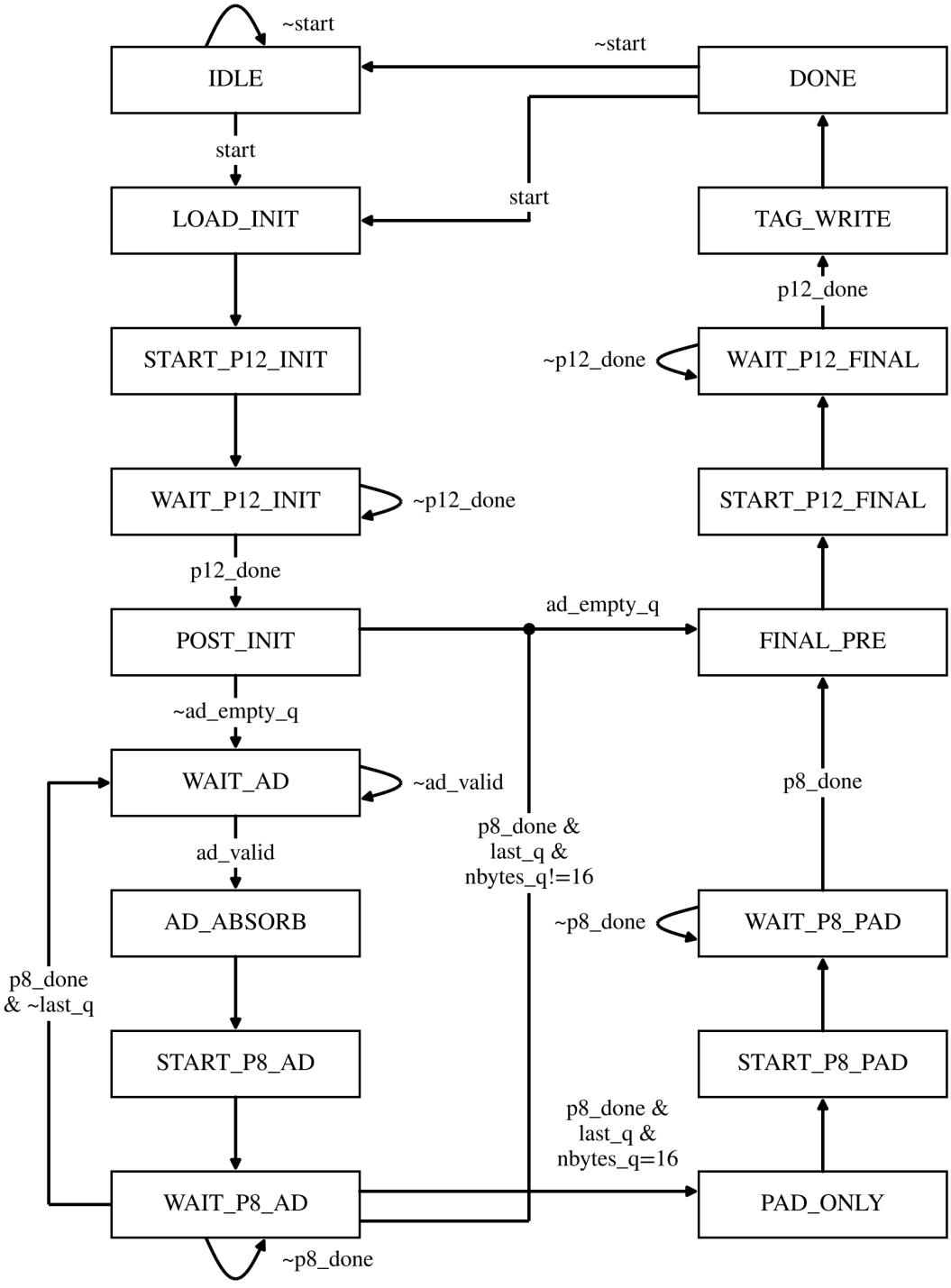
Port	Dir	Width	Data Type	Action level	Mô tả
Xung nhịp và đặt lại					
`clk`	Input	1	wire		Xung nhịp hệ thống.
`rst_n`	Input	1	wire	Mức thấp	Đặt lại không đồng bộ/đồng bộ theo thiết kế, đưa controller về trạng thái đầu.
Điều khiển					
`start`	Input	1	wire	Mức cao	Yêu cầu bắt đầu tính tag cho một khối AD.
`ad_empty`	Input	1	wire	Mức cao	Báo AD rỗng. Trong SoC nối mức 0 vì AD của khối lệnh luôn khác rỗng.
`busy`	Output	1	wire	Mức cao	Khối đang tính.
`done`	Output	1	wire	Mức cao	Tag đã sẵn sàng trên `tag_out`.
Khóa và nonce					
`key_d`	Input	128	wire [127:0]	—	Khóa 128 bit, nạp vào `ascon_key_nonce_reg` (tách thành K0, K1).

Port	Dir	Width	Data Type	Action level	Mô tả
nonce_d	Input	128	wire [127:0]	—	Nonce 128 bit, nạp vào ascon_key_nonce_reg (tách thành N0, N1).
Dữ liệu AD (giao tiếp valid/ready)					
ad_data	Input	128	wire [127:0]	—	Khối AD đưa vào ascon_ad_buffer_padding.
ad_nbytes	Input	5	wire [4:0]	—	Số byte hợp lệ của khối AD hiện tại.
ad_last	Input	1	wire	Mức cao	Khối AD hiện tại là khối cuối.
ad_valid	Input	1	wire	Mức cao	Dữ liệu AD đang hợp lệ.
ad_ready	Output	1	wire	Mức cao	Controller sẵn sàng nhận khối AD.
Kết quả					
tag_out	Output	128	wire [127:0]	—	Tag Ascon-128a, hợp lệ khi done=1.



Hình 6: Sơ đồ khối Ascon-128a

Sơ đồ thiết kế



Hình 7: Máy trạng thái FSM của khối `ascon_controller`

Máy trạng thái FSM Ascon-128a

3.3 Khối System Memory

3.3.1 Chức năng

Khối `system_memory` là bộ nhớ chính, nằm sau `authenticated_memory_wrapper`. Khối tổ chức ba vùng nhớ độc lập trên cùng một không gian địa chỉ 32 bit và phục vụ từng yêu cầu truy cập theo giao thức `mem_valid`/`mem_ready`:

- **CODE** (`0x0000_0000`–`0x0000_1FFF`): 2048 từ 32 bit (8 KiB), chỉ đọc, chứa chương trình của CPU.
- **TAG** (`0x0001_0000`–`0x0001_1FFF`): 2048 từ 32 bit (8 KiB), chỉ đọc, chứa các tag Ascon-128a tương ứng với từng khối lệnh trong vùng CODE.
- **DATA** (`0x0002_0000`–`0x0002_0FFF`): 1024 từ 32 bit (4 KiB), đọc/ghi, chứa dữ liệu của chương trình. Vùng này hỗ trợ ghi theo từng byte nhờ `mem_wstrb[3:0]`.

Với mỗi yêu cầu, khối thực hiện các việc sau:

1. **Giải mã địa chỉ.** Một yêu cầu được xem là trúng vùng nhớ khi địa chỉ căn chỉnh theo từ (`mem_addr[1:0]=00`) và nằm trong khoảng địa chỉ của vùng đó. Chỉ số từ trong mảng được lấy bằng cách trừ địa chỉ cơ sở của vùng rồi dịch phải 2 bit.
2. **Đọc.** Khi `mem_wstrb=0000`, khối trả về từ nhớ tại vùng trúng qua `mem_rdata`.
3. **Ghi.** Khi `mem_wstrb`$\neq$0000 và địa chỉ thuộc vùng DATA, khối ghi các byte được chọn bởi `mem_wstrb` vào từ tương ứng. Vùng CODE và TAG không có đường ghi nên các lệnh ghi vào hai vùng này không làm thay đổi nội dung.
4. **Phản hồi.** Khối luôn trả lời mọi yêu cầu bằng một xung `mem_ready` ở chu kỳ kế tiếp sau khi nhận `mem_valid`. Yêu cầu ngoài vùng nhớ, địa chỉ không căn chỉnh hoặc lệnh ghi đều trả `mem_rdata=0`.

Bảng 6: Bản đồ bộ nhớ trong RTL hiện tại

Vùng	Địa chỉ đầu	Địa chỉ cuối	Dung lượng
CODE	`0x00000000`	`0x00001FFF`	8 KiB
TAG	`0x00010000`	`0x00011FFF`	8 KiB
DATA	`0x00020000`	`0x00020FFF`	4 KiB

3.3.2 Thuật toán và nguyên lý vận hành

Thuật toán xử lý một yêu cầu. Yêu cầu được chấp nhận ở cạnh lên xung nhịp khi `mem_valid`=1 và `mem_ready`=0:

1. Đặt `mem_ready`←1 và `mem_rdata`←0 (giá trị mặc định).
2. Nếu `mem_wstrb`=0000 (đọc): `mem_rdata` nhận từ nhớ của vùng trúng (`code_rom`, `tag_rom` hoặc `data_ram`). Nếu không vùng nào trúng thì giữ 0.
3. Nếu `mem_wstrb`≠0000 và `data_hit`=1 (ghi): lưu chỉ số từ, dữ liệu và mặt nạ byte vào `write_index_q`, `write_data_q`, `write_strb_q`, đồng thời đặt `write_pending`=1.
4. Ở chu kỳ kế tiếp (`mem_ready`=1, `write_pending`=1), nếu `mem_valid` còn giữ ở mức 1 thì khối ghi từng byte vào `data_ram` theo `write_strb_q[i]`. Cuối chu kỳ này `mem_ready` và `write_pending` về 0.

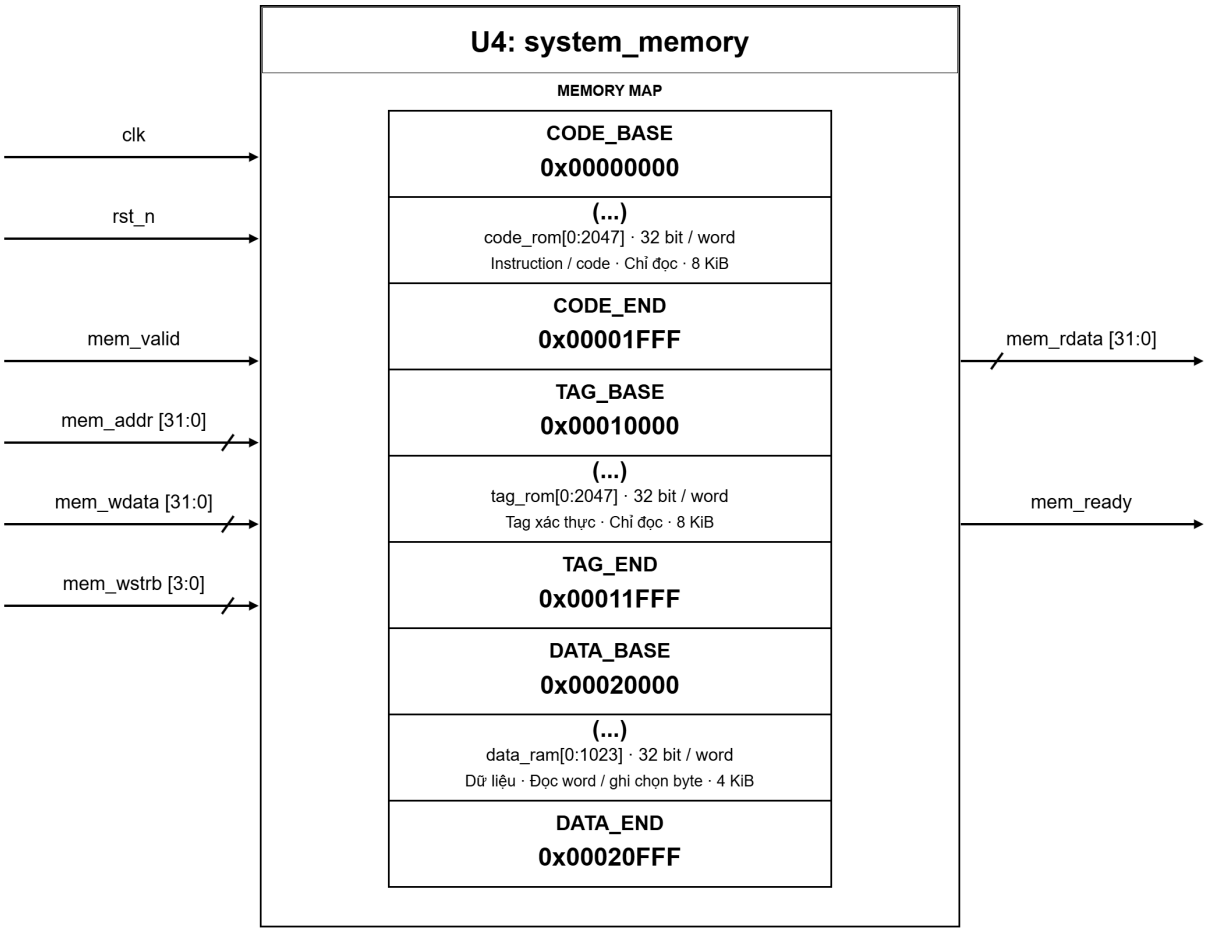
Lệnh ghi ngoài vùng DATA, ghi vào CODE hoặc TAG, hoặc địa chỉ không căn chỉnh không thay đổi bộ nhớ, nhưng vẫn được phản hồi `mem_ready` và `mem_rdata`=0.

Giao thức và thời gian. Giao thức là bắt tay một xung. `mem_ready` lên 1 đúng một chu kỳ sau chu kỳ nhận yêu cầu, cùng lúc `mem_rdata` có giá trị hợp lệ. Bên yêu cầu giữ nguyên `mem_valid` và các tín hiệu địa chỉ, dữ liệu cho đến khi thấy `mem_ready`. Một lần truy cập gồm chu kỳ nhận (t_0) và chu kỳ trả lời (t_1); yêu cầu kế tiếp chỉ được nhận từ t_2 trở đi vì ở t_1 `mem_ready`=1. Với lệnh ghi, dữ liệu được ghi vào mảng ở cuối t_1 nên đọc lại cùng địa chỉ ở yêu cầu kế tiếp luôn thấy giá trị mới.

Bảng 7: Hành vi của `system_memory` theo loại truy cập

Địa chỉ	**Đọc** (`wstrb`=0)	**Ghi** (`wstrb`≠0)	**Ghi chú**
CODE `0x0000_0000-1FFF`	trả từ `code_rom`	bỏ qua, `rdata`=0	chỉ đọc
TAG `0x0001_0000-1FFF`	trả từ `tag_rom`	bỏ qua, `rdata`=0	chỉ đọc
DATA `0x0002_0000-0FFF`	trả từ `data_ram`	ghi theo byte	đọc/ghi
Ngoài vùng hoặc lệch từ	`rdata`=0	bỏ qua	vẫn có `mem_ready`

3.3.3 Thiết kế phần cứng



Hình 8: Sơ đồ System Memory

3.4 Khối Trusted Key and Context

3.4.1 Chức năng

Khối `trusted_key_and_context` cấp cho wrapper hai thông tin cố định của hệ thống: khóa xác thực mã `K_code` (128 bit) và vector cấu hình `CFG` (224 bit). Khóa được đưa tới cổng `key_d` của `ascon_top` để tính tag. `CFG` được `auth_cfg_decode` tách thành các trường định danh chương trình, ảnh chương trình và ranh giới các vùng nhớ. hay cổng vào.

3.4.2 Tổ chức khóa và ngữ cảnh

`CFG` được ghép từ năm tham số theo thứ tự từ bit cao xuống bit thấp:

Bảng 8: Tổ chức vector CFG 224 bit

Trường	Bit	Độ rộng	Ý nghĩa
PROGRAM_ID	[223:160]	64	Định danh chương trình
IMAGE_ID	[159:96]	64	Định danh ảnh (image) chương trình
CODE_BASE	[95:64]	32	Địa chỉ đầu vùng CODE
CODE_LIMIT	[63:32]	32	Địa chỉ cuối vùng CODE
TAG_BASE	[31:0]	32	Địa chỉ đầu vùng TAG

PROGRAM_ID và IMAGE_ID gắn tag với một chương trình và một ảnh cụ thể, còn CODE_BASE, CODE_LIMIT, TAG_BASE xác định vùng lệnh và vùng tag mà wrapper kiểm tra địa chỉ. Khối auth_cfg_decode tách CFG theo đúng bố trí bit ở Bảng 8.

Khóa K_code hiện được cung cấp qua tham số KEY_CODE trong RTL, nghĩa là khóa được cố định lúc tổng hợp và nằm trong bitstream. Cách này phù hợp với mức nguyên mẫu nhưng chưa phải kho khóa an toàn (xem hạn chế bên dưới).

3.4.3 Thiết kế phần cứng

Bảng 9: Giao diện tín hiệu của khối trusted_key_and_context

Port	Dir	Width	Data Type	Action level	Mô tả
K_code	Output	128	wire [127:0]	—	Khóa xác thực mã, bằng KEY_CODE.
CFG	Output	224	wire [223:0]	—	Vector cấu hình {PROGRAM_ID, IMAGE_ID, CODE_BASE, CODE_LIMIT, TAG_BASE}.

Bảng 10: Tham số của khối trusted_key_and_context

Tham số	Độ rộng	Giá trị mặc định
KEY_CODE	128	128'h000102030405060708090A0B0C0D0E0F
PROGRAM_ID	64	64'd1
IMAGE_ID	64	64'd1
CODE_BASE	32	32'h00000000
CODE_LIMIT	32	32'h00001FFF
TAG_BASE	32	32'h00010000

Khối chỉ gồm hai phép gán: K_code=KEY_CODE và CFG={PROGRAM_ID, IMAGE_ID, CODE_BASE, CODE_LIMIT, TAG_BASE}, không tốn tài nguyên logic ngoài dây nối. Trong auth_soc_top, chỉ

KEY_CODE, PROGRAM_ID và IMAGE_ID được ghi đè từ tham số của top; ba tham số địa chỉ giữ giá trị mặc định, trùng với bản đồ địa chỉ của system_memory.

Hạn chế. Khóa nằm sẵn trong RTL nên ai đọc được bitstream hoặc mã nguồn đều biết khóa. Đây là giả định của nguyên mẫu. Một hệ thống thật cần nguồn khóa an toàn, ví dụ eFUSE, PUF hoặc bộ nhớ khóa được bảo vệ.

3.4.4 Thiết kế phần cứng

[Cần bổ sung: Port map, giá trị cấu hình, đường cấp khóa trực tiếp tới Ascon và các giả định tin cậy đối với khóa/ngữ cảnh.]

3.5 Lõi PicoRV32 và giao diện tích hợp

4 Lõi PicoRV32 và giao diện tích hợp

4.1 Chức năng

PicoRV32 [1] là lõi CPU RISC-V RV32I mã nguồn mở của YosysHQ (giấy phép ISC), được dùng để thực thi firmware và kiểm tra hoạt động của cơ chế bảo vệ lệnh. Lõi được dùng nguyên bản, không sửa đổi. Toàn bộ cơ chế xác thực nằm ở authenticated_memory_wrapper, đặt giữa CPU và bộ nhớ: CPU chỉ thấy một bộ nhớ thông thường qua giao diện bộ nhớ gốc của PicoRV32, còn việc kiểm tra tag của từng khối lệnh được thực hiện trong suốt phía sau giao diện đó.

4.2 Cấu hình sử dụng

Trong auth_soc_top, PicoRV32 được khởi tạo với các tham số ở Bảng 11. Chỉ STACKADDR khác giá trị mặc định của lõi; các tham số còn lại được ghi tường minh để cấu hình rõ ràng.

Bảng 11: Tham số PicoRV32 trong `auth_soc_top`

Tham số	**Giá trị**	**Mặc định**	**Ý nghĩa**
`COMPRESSED_ISA`	0	0	Tắt tập lệnh nén; mọi lệnh dài 4 byte, căn chỉnh theo từ.
`CATCH_ILLINSN`	1	1	Lệnh không hợp lệ đưa CPU vào trạng thái `trap`.
`CATCH_MISALIGN`	1	1	Truy cập lệch địa chỉ đưa CPU vào `trap`.
`ENABLE_IRQ`	0	0	Tắt ngắt; cổng `irq` nối 0.
`PROGADDR_RESET`	`0x0000_0000`	`0x0000_0000`	Địa chỉ bắt đầu sau reset, trùng `CODE_BASE`.
`STACKADDR`	`0x0002_1000`	`0xFFFF_FFFF`	Giá trị nạp vào con trỏ ngăn xếp `sp` (`x2`) khi reset, ngay sau vùng DATA (`0x20000`–`0x20FFF`). Mặc định là tắt.

Các cổng PCPI (`pcpi_wr`, `pcpi_rd`, `pcpi_wait`, `pcpi_ready`) nối mức 0 vì không dùng bộ đồng xử lý. Phép nhân và chia không được bật (`ENABLE_MUL`=`ENABLE_DIV`=0), nên firmware phải biên dịch cho RV32I thuần; lệnh nhân/chia sẽ bị coi là lệnh không hợp lệ. Các tham số còn lại giữ giá trị mặc định của lõi.

4.3 Giao diện và hành vi khi xác thực thất bại

Giao diện bộ nhớ. PicoRV32 giao tiếp qua giao diện bộ nhớ gốc kiểu valid–ready, mỗi lúc thực hiện một giao dịch. Các tín hiệu được liệt kê ở Bảng 12.

Bảng 12: Giao diện bộ nhớ của PicoRV32 nối vào wrapper

Tín hiệu	**Hướng (từ CPU)**	**Rộng**	**Ý nghĩa**
`mem_valid`	Output	1	CPU có yêu cầu; giữ ở mức cao đến khi nhận `mem_ready`.
`mem_instr`	Output	1	1 khi nạp lệnh, 0 khi truy cập dữ liệu. Wrapper dùng bit này để chỉ xác thực lệnh.
`mem_addr`	Output	32	Địa chỉ, luôn căn chỉnh theo từ (hai bit thấp bằng 0).
`mem_wdata`	Output	32	Dữ liệu ghi.
`mem_wstrb`	Output	4	Bằng 0000 là đọc; khác 0000 là ghi và mỗi bit chọn một byte. Chỉ có 8 giá trị: 0000, 1111, 1100, 0011, 1000, 0100, 0010, 0001.
`mem_ready`	Input	1	Wrapper báo giao dịch hoàn tất.
`mem_rdata`	Input	32	Dữ liệu đọc; phải hợp lệ ở chu kỳ có `mem_ready`.
`trap`	Output	1	CPU dừng do lệnh không hợp lệ hoặc truy cập lệch địa chỉ; giữ ở mức 1 đến khi reset.

CPU chờ trong khi xác thực. CPU không biết có bước xác thực: nó chỉ giữ `mem_valid` và các tín hiệu địa chỉ/dữ liệu cho đến khi thấy `mem_ready`. Khi nạp một lệnh trúng vùng đệm của wrapper (hit), `mem_ready` được trả sau khoảng 3 chu kỳ. Khi trượt (miss), wrapper chưa trả `mem_ready`; nó đọc khối lệnh và tag từ `system_memory`, để Ascon tính tag rồi so sánh, và chỉ trả lệnh cho CPU khi tag khớp. Trong khoảng thời gian đó CPU đứng chờ và không có bộ đếm thời gian chờ nào ở phía CPU. Chi tiết số chu kỳ xem Mục 3.1.

Khi xác thực thất bại. Wrapper phát hiện lỗi trong ba trường hợp: tag không khớp, nạp lệnh ngoài vùng CODE, và ghi vào vùng được bảo vệ. Khi đó wrapper chốt cờ `auth_fault` (giữ ở mức 1 đến khi reset) và xử lý như sau:

- **Nạp lệnh bị từ chối.** Với `FAULT_HALT`=0, wrapper vẫn trả `mem_ready` nhưng `mem_rdata`=0, nên lệnh thật không bao giờ tới CPU. Từ `0x00000000` không phải lệnh RV32I hợp lệ, vì vậy PicoRV32 (với `CATCH_ILLINSN`=1) đặt `trap`=1 và dừng. Với `FAULT_HALT`≠0, wrapper không trả `mem_ready`, CPU bị treo.
- **Ghi vào vùng được bảo vệ.** Lệnh ghi bị chặn, không tới `system_memory`. Lệnh ghi không đọc dữ liệu trả về nên CPU không vào `trap`; chỉ `auth_fault` báo lỗi.

Hai tín hiệu `trap` và `auth_fault` là hai cổng ra của `auth_soc_top`, dùng để quan sát kết quả trong mô phỏng và trên board.

5 MÔ HÌNH KIỂM CHỨNG

5.1 Mô hình kiểm chứng tổng quan

5.2 Mô hình tham chiếu và luồng tạo code/tag

5.3 Testbench, monitor và cơ chế đối sánh

5.4 Kế hoạch kiểm chứng và tiêu chí đánh giá

6 KIỂM CHỨNG

6.1 Kiểm chứng các khối con của Wrapper

6.2 Kiểm chứng khối Ascon-128a

6.3 Kiểm chứng khối System Memory

6.4 Kiểm chứng Authenticated Memory Wrapper

6.5 Kiểm chứng tích hợp PicoRV32 và các kịch bản tấn công

- Firmware và tag hợp lệ.
- Sửa code hoặc sửa tag.
- Đổi chỗ block cùng tag sang địa chỉ khác.
- Tag tạo với PROGRAM_ID, IMAGE_ID hoặc khóa khác.
- Nhảy vào vùng DATA hoặc firmware chưa ký.
- Sửa memory sau khi block đã vào buffer xác thực.
- CPU ghi vào vùng code được bảo vệ.
- Firmware C hợp lệ và firmware C bị sửa.

6.6 Tổng hợp kết quả kiểm chứng

Bảng 13: Tổng hợp kết quả kiểm chứng

Nhóm kiểm chứng	**Số test**	**PASS**	**FAIL**
Khối con của Wrapper	[Điền]	[Điền]	[Điền]
Ascon-128a	[Điền]	[Điền]	[Điền]
System Memory	[Điền]	[Điền]	[Điền]
Authenticated Memory	[Điền]	[Điền]	[Điền]
PicoRV32 tích hợp	[Điền]	[Điền]	[Điền]

7 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

7.1 Môi trường và cấu hình thực nghiệm

7.2 Kết quả chức năng và phát hiện sửa đổi mã lệnh

7.3 Hiệu năng và chi phí xác thực

$$\text{Overhead}_{\text{cycle}} = \frac{C_{\text{auth}} - C_{\text{direct}}}{C_{\text{direct}}} \times 100\%. \tag{12}$$

7.4 Ảnh hưởng của kích thước block và độ dài tag

Bảng 14: So sánh cấu hình trên cùng firmware

Cấu hình	**G**	**T**	**Chu kỳ**	**Số verify**
CPU nối trực tiếp memory	–	–	[Điền]	–
Authenticated	1	32	[Điền]	[Điền]
Authenticated	4	128	[Điền]	[Điền]
Authenticated	8	128	[Điền]	[Điền]

Chi phí lưu tag trên dung lượng code được bảo vệ:

$$\text{Metadata overhead} = \frac{T}{32G} \times 100\%. \tag{13}$$

7.5 Tài nguyên, timing và công suất

7.6 Đánh giá tổng hợp và giới hạn

Tài liệu

[1] C. Wolf, "PicoRV32 – a size-optimized RISC-V CPU." GitHub repository, YosysHQ. Truy câØ: 2026.

[2] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," *Journal of Cryptology*, vol. 34, no. 3, 2021.

[3] National Institute of Standards and Technology, "NIST SP 800-232: Ascon-based lightweight cryptography standards for constrained devices," tech. rep., NIST, 2025.